

区块链技术的应用

智能合约与以太坊

王勇超

2026.5.19

本讲课程目标



1. 理解 **智能合约** 的本质与运行逻辑



2. 掌握 **以太坊 (Ethereum)** 的核心机制

回顾与引入

⚠ 比特币的局限性

- 上节课我们讲到，比特币虽然是伟大的发明，但它本质上只是一个**去中心化的账本**。
- 它的脚本语言非常简单，**只能用来转账**，无法在上面编写复杂的应用程序。

什么是智能合约？

智能合约 (Smart Contract) 本质上就是
运行在区块链上的代码。

它是一套以数字形式定义的承诺，当满足预设条件时程序会**自动执行**，无需第三方干预。

```
if (条件满足) {  
    执行转账或操作();  
}
```

案例：自动售货机

最早的“智能合约”

- 🤖 无需售货员，机器已写好**规则**。
- 🤖 规则：投入3元且按下可乐，掉出一罐**可乐**。
- 🤖 条件满足后，机器**自动执行**。

```
if (money >= 3 AND button == "Cola") {  
    dispense("Cola");  
}
```


智能合约的特点



1. 自动执行

条件一旦满足，代码**自动触发**，无需人工干预。



2. 不可篡改

代码部署到区块链后，**任何人（包括开发者）都无法修改**。



3. 公开透明

合约的规则和执行过程对全网公开，**完全透明可查**。

以太坊 (Ethereum)

◆ 世界计算机

- 以太坊是一个开源的、全球去中心化的计算架构。
- 它不仅是一个账本，更是一台 **“永不宕机的世界计算机”**。
- 任何人都可以利用以太坊网络创建和运行**智能合约**和去中心化应用（DApp）。

以太坊 vs 比特币



比特币

- 定位：去中心化账本
- 功能：只能进行转账交易
- 脚本：非图灵完备（简单）
- 比喻：像一个**计算器**，只能做特定计算



以太坊

- 定位：去中心化计算平台
- 功能：运行智能合约和DApp
- 脚本：图灵完备（复杂）
- 比喻：像一部**智能手机**，可以安装各种App

什么是图灵完备？

1. 条件判断 (if 分支)

能根据不同条件判断结果走不同流程

2. 循环执行 (while / 递归)

可重复执行代码，能无限迭代

3. 可以读写存储数据

有变量、内存、状态，能存数据改数据

EVM (以太坊虚拟机)

智能合约的运行环境

EVM (Ethereum Virtual Machine) 是以太坊的“**操作系统**”，负责执行智能合约的代码，并确保在全网所有节点上，代码的执行结果**完全一致**。

智能合约代码



EVM 编译执行



状态更新上链

EVM运行机制

以太坊全网每一个全节点，都内置独立 EVM 虚拟机。

- 所有以太坊全节点本地都自带一套 EVM。
- 交易广播全网后，每个节点都用自己本机的 EVM 独立执行一遍合约。
- 所有节点执行结果一模一样，最终达成共识、打包区块。

EVM 核心特点

1. 隔离安全

合约代码只在 EVM 内运行，不能直接操作系统，防攻击。

2. 图灵完备

支持循环、判断、递归，能跑任意智能合约逻辑。

3. Gas 计费机制

每一步运算都扣 Gas，防止无限循环卡死整条链。

天下没有免费的午餐

在以太坊上运行代码，必须付费。

Gas 机制

以太坊的“燃料”

在以太坊上，无论是转账还是运行智能合约，都需要消耗计算资源。

为了衡量这些资源的消耗以太坊引入了 **Gas (燃料)** 机制。

简单来说：**运行代码需要付钱**。越复杂的操作，消耗的 Gas 越多。

案例：创业做产品

通俗理解 Gas 费

- 你（用户）创业招募员工（**矿工**）开发产品（**执行合约**）。
- 产品越复杂（**代码越复杂**），消耗的资金（**Gas**）就越多。
- 如果资金不足，产品开发一半就会停下（**Out of Gas**），而且**已经投入的资金（员工工资）不可能退回！**

为什么需要 Gas?

1. 防止死循环



图灵完备语言可能出现死循环，Gas耗尽程序就会**强制停止**。

2. 防止滥用资源



天下没有免费的午餐，收费可以有效防止恶意攻击者**瘫痪网络**。

3. 激励矿工



支付的 Gas 费会作为奖励发给打包区块的**矿工（验证者）**。

以太坊的账户模型



外部账户 (EOA)

- 由**私钥**控制
- 普通用户使用的账户
- 可以发起交易
- 没有代码



合约账户 (CA)

- 由**代码**控制
- 智能合约的账户
- 只能被动接收交易触发
- 包含代码和状态

链上交互流程

运行原理

1. 只能由 EOA (外部账户) 发起交易;
2. 交易发给 CA (合约账户) ;
3. 合约账户收到请求 → EVM 执行代码;
4. 合约可以转钱、改数据、调用其他合约;
5. 执行结束, 全网同步状态。

Solidity 语言

Solidity 是一门面向合约、静态类型的高级编程语言，专门用于编写以太坊及兼容 EVM 公链上的智能合约。

- 开发团队：以太坊基金会
- 语法风格：类似 JavaScript + Java + C++
- 运行环境：EVM 以太坊虚拟机
- 主流版本：0.8.x（最稳定、最常用）

```
// 指定编译器版本
pragma solidity ^0.8.20;

// 定义合约
contract Counter {
    // 状态变量：链上存储数字
    uint public count;

    // 累加函数
    function add() public {
        count += 1;
    }

    // 减法函数
    function sub() public {
        count -= 1;
    }
}
```

例1：简单计数器

- uint public count：公开整型变量，链上永久保存。
- 调用add、sub会修改数据，消耗**Gas**。
- 外部可直接读取 count 数值。

```
pragma solidity ^0.8.20;

contract ReceiveETH {
    // 定义事件：记录收款信息
    event ReceiveLog(address from, uint amount);

    // 接收ETH专用函数
    receive() external payable {
        // 触发事件记录日志
        emit ReceiveLog(msg.sender, msg.value);
    }

    // 查询合约余额
    function getBalance() public view returns(uint) {
        return address(this).balance;
    }
}
```

例 2：转账收款 + 事件日志

1. payable 标识函数可以接收以太币
2. msg.sender = 转账人地址
3. msg.value = 转入金额（单位 wei）
4. view 只读查询，免费不耗 Gas
5. 事件用于前端监听交易记录

```
pragma solidity ^0.8.20;

contract SetInfo {
    string public userName;

    // 设置用户名
    function setName(string memory _name) public {
        userName = _name;
    }
}
```

例 3：传参赋值（常用业务模板）

- memory：临时存储，函数执行完释放
- 传入参数修改链上状态变量，最基础业务写法

Solidity 语言

运行原理

1. 开发者编写 Solidity 源码
2. 编译成 EVM 字节码
3. 部署上链，存入合约账户
4. 用户通过钱包发起交易调用合约
5. 全网节点 EVM 统一执行，结果全网共识

DApp (去中心化应用)



DApp (Decentralized Application) 在用户看来, 和普通的App或网站没有区别。

唯一的区别是: 它的后端逻辑不是运行在中心化服务器上, 而是运行在**区块链 (智能合约)** 上。

案例：去中心化金融 (DeFi)

没有银行的金融服务

 传统金融：借贷、交易都需要通过**银行或交易所**等中介。

 DeFi：所有的金融规则都写在**智能合约**里。

 只要你抵押了足够的资产，合约就会**自动放款**，无需审批，没有人工干预。

代码即法律

Code is Law

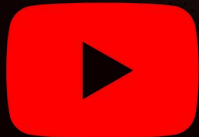
智能合约的风险

代码漏洞可能导致巨额损失

- ⚠ 智能合约一旦部署，代码**不可篡改**。
- ⚠ 如果代码存在漏洞（Bug），黑客可以利用漏洞**盗取资产**。
- ⚠ 著名案例：The DAO 事件（黑客盗取了价值数千万美元的以太币）。
- ⚠ 结论：在区块链上，**安全审计**比传统软件开发更重要。

拓展：以太坊发展史

以太坊是如何诞生的？



了解天才少年 **Vitalik Buterin (V神)** 的故事。

 <https://www.zhihu.com/zvideo/1704813091676700673>

本讲知识要点

智能合约



运行在区块链上的**自动执行代码**，代码即法律。

EVM



以太坊虚拟机，智能合约的**运行环境**。

DApp



去中心化应用，**前端界面 + 智能合约**。

以太坊



支持智能合约的**去中心化计算平台**（世界计算机）。

Gas 机制



运行代码需要消耗的**燃料费**，防止资源滥用。